

INTERVIEW PREPARATION GUIDE

TaskVoice

An end-to-end guide to the product, architecture, code, decisions, trade-offs, and interview questions

YOUR ONE-LINE EXPLANATION TaskVoice is a local-first React Native to-do app that supports typed and spoken task capture, persists tasks with AsyncStorage, and securely turns natural-language recordings into one or more structured tasks through a small Node backend.

Built for the July 2026 AAIR Labs Developer Exercise
Expo 54 • React Native • TypeScript • Express • Groq • Jest

How to use this handbook

Read Sections 1–5 to understand the full system. Use Sections 6–9 to prepare your technical explanation. Rehearse the questions in Section 10 and the demonstration script in Section 11.

Section	What you should learn
1. Product and brief	What was required and what the app delivers.
2. Architecture	How mobile, storage, backend, and AI fit together.
3. Task lifecycle	How add, edit, complete, delete, filter, and persistence work.
4. Voice lifecycle	Every step from FAB press to structured tasks.
5. Code structure	What each folder and important file owns.
6–9. Decisions	Why the implementation was designed this way.
10. Interview Q&A	Likely questions and confident answers.
11. Demo plan	A concise, reliable live walkthrough.

INTERVIEW PRINCIPLE Explain the problem first, then the data flow, then the implementation detail. Do not begin by listing libraries.

1. Product, requirements, and value

What TaskVoice does

TaskVoice helps a user capture and manage personal tasks. A user can add a title and optional description, complete or reopen a task, delete it, search and filter the list, assign a due date, switch themes, and edit both manual and voice-created tasks.

How it satisfies the brief

Brief requirement	Implementation
Add tasks	Reusable TaskFormScreen validates title and accepts description/due date.
Complete/incomplete	TOGGLE_TASK changes completed and updatedAt.
Delete	TaskItem requests native confirmation before deletion.
View all tasks	TaskListScreen renders a FlatList of derived tasks.
Persistence	TaskProvider loads and saves through AsyncStorage.
Two-screen navigation	React Navigation connects Tasks and AddTask; EditTask reuses the same form.
Voice FAB	FloatingActionButton starts Expo Audio recording.
Speech transcription	Backend sends audio to Groq-hosted whisper-large-v3.
Multiple tasks	LLM extraction returns a structured array and deduplicates titles.
Screenshots	Physical-iPhone screenshots and an MP4 demo are included.

Bonus work

- Due dates, relative deadline badges, and due-date sorting.
- Search plus All, Active, and Done filters with counts.
- Persisted light and dark themes.
- Strict TypeScript, reducer/unit tests, ESLint, and Prettier.
- Professional modal transitions, error states, accessibility labels, and physical-device evidence.

2. System architecture

ARCHITECTURE SUMMARY The mobile app owns interaction and local state. AsyncStorage owns device persistence. The Express server owns secrets and AI calls. Groq performs transcription and structured extraction.

Layered view

Layer	Responsibility	Key files
Presentation	Screens, task cards, modals, filtering controls	src/screens, src/components
Application state	Commands and deterministic task transitions	TaskContext.tsx, taskReducer.ts
Device services	AsyncStorage and voice API client	taskStorage.ts, voiceTaskApi.ts
Domain utilities	Dates, validation, normalization, IDs	src/utlis
Navigation/theme	Routes and semantic palettes	AppNavigator.tsx, ThemeContext.tsx
Backend API	Upload validation, orchestration, safe errors	server/src/routes
AI services	Transcription and structured extraction	server/src/services

End-to-end dependency flow

```

User
  ■■ manual input → TaskFormScreen → TaskContext → taskReducer → AsyncStorage
  ■■ voice FAB → useVoiceInput → voiceTaskApi → Express route
                    ■■ transcribeAudio → Whisper
                    ■■ extractTasks → structured LLM output
                        response → normalize → TaskContext → AsyncStorage
  
```

Why this architecture is proportionate

The app is small, so Redux and a database would add ceremony without solving a real scaling problem. Context plus useReducer makes state transitions explicit and testable. A narrow backend is necessary because API keys must never be shipped in a mobile bundle.

3. Application boot and task lifecycle

App startup

1. App mounts ThemeProvider, which chooses the system theme and restores a saved preference.
2. TaskProvider starts with isHydrated=false and requests saved tasks.
3. TaskListScreen shows a loading state until hydration finishes.
4. After a successful load, tasks are dispatched through LOAD_TASKS.
5. Only then may automatic saving begin.

CRITICAL PERSISTENCE INVARIANT The loadedSafely ref prevents the initial empty reducer state from overwriting existing device data when loading fails. Hydration state alone is insufficient because the error action also ends loading.

Task data model

```
interface Task {  
  id: string;  
  title: string;  
  description?: string;  
  dueDate?: string;    // local YYYY-MM-DD  
  completed: boolean;  
  source: 'manual' | 'voice';  
  createdAt: string;  
  updatedAt: string;  
}
```

Reducer actions

Action	Effect
LOAD_TASKS	Replaces state with validated stored tasks and ends hydration.
ADD_TASKS	Prepends one or more new tasks.
UPDATE_TASK	Changes editable fields while preserving ID, source, completion, and creation time.
TOGGLE_TASK	Flips completion and updates the timestamp.
DELETE_TASK	Removes exactly the matching ID.
STORAGE_ERROR	Exposes a user-facing persistence error.

4. Manual task flows

Add task

6. The + button navigates to AddTask.
7. TaskFormScreen initializes empty controlled fields.
8. validateTaskTitle rejects whitespace-only input.
9. TaskContext creates an ID and timestamps, trims values, and dispatches ADD_TASKS.
10. The persistence effect observes the new state and saves the full array.
11. navigation.goBack returns to the updated list.

Edit task

TaskItem passes its ID through the EditTask route. TaskFormScreen finds the task and pre-fills the same controls used for creation. UPDATE_TASK changes title, description, and due date while deliberately preserving source and completed. Assigning optional fields as undefined allows users to clear them.

Complete and delete

- Completion uses a checkbox-like Pressable with accessibilityState.checked.
- Deletion uses Alert.alert with cancel and destructive choices.
- Both changes flow through the reducer and are persisted by the same effect.

Search, filters, and sorting

TaskListScreen derives visible tasks with useMemo. Search normalizes the query and searches title plus description. Filters do not mutate data; they select All, Active, or Done. sortTasks always copies the array before sorting, preventing accidental state mutation.

WHY DERIVED STATE The app stores only canonical tasks. Query, filter, counts, and sorted results are calculated from tasks, avoiding synchronization bugs.

5. Voice flow: mobile to AI and back

Mobile recording state machine

State	What the user sees	What the code does
idle	Normal task list	Recorder is inactive.
listening	Listening sheet and Stop button	Expo Audio records high-quality audio.
processing	Spinner and Creating your tasks	File is uploaded; dismissal is blocked.
success	Tasks added confirmation	Structured tasks have entered state.
error	Actionable error and Done	User can recover without losing existing tasks.

Request path

12. FAB calls `useVoiceInput.start`.
13. The hook requests microphone permission and configures recording audio mode.
14. Stop finalizes the recording and exposes `recorder.uri`.
15. `voiceTaskApi` creates multipart `FormData` under the audio field.
16. `POST /api/voice-tasks` sends the recording to the local backend.
17. The backend validates file presence, MIME type, file count, and size.
18. `transcribeAudio` sends the file to `whisper-large-v3`.
19. `extractTasks` asks the LLM for strict JSON matching a task-list schema.
20. The server trims and deduplicates results and returns transcript plus tasks.
21. The client normalizes the untrusted response before adding tasks.

Fallback behavior

Transcription is the minimum successful result. If structured extraction fails or returns no tasks, the transcript becomes one task and `fallback=true` is returned. This is graceful degradation: an optional AI enhancement cannot erase useful user input.

6. Backend and API design

Why a backend is required

Mobile bundles are inspectable. Embedding GROQ_API_KEY in EXPO_PUBLIC variables would expose it to every user. The server reads the secret from server/.env and the client knows only a public backend URL.

API contract

```
POST /api/voice-tasks
Content-Type: multipart/form-data
Field: audio

{
  "transcript": "Buy provisions and call Mum",
  "tasks": [{"title": "Buy provisions"}, {"title": "Call Mum"}]
}
```

Responsibilities by backend file

File	Responsibility
server/src/index.ts	Loads environment, creates the Groq-compatible OpenAI client, mounts routes, starts server.
routes/voiceTasks.ts	Upload limits, validation, orchestration, fallbacks, HTTP status codes.
services/transcribeAudio.ts	Converts Buffer to an SDK file and requests transcription.
services/extractTasks.ts	Prompts for explicit actions, enforces JSON schema, trims and deduplicates.

Security posture

- API key remains server-side and .env files are ignored by Git.
- x-powered-by is disabled.
- Uploads are memory-buffered but size- and count-limited.
- Errors returned to clients are generic; internal messages are logged.
- Production follow-ups: authentication, rate limiting, restricted CORS, request IDs, telemetry, and streaming storage.

7. Navigation, theme, dates, and UI composition

Navigation

AppNavigator uses a typed native stack. Tasks is the root screen. AddTask and EditTask both render TaskFormScreen with different titles and parameters. Reuse prevents the two forms from drifting.

Theme system

ThemeContext maps a light/dark mode to semantic tokens such as background, surface, ink, muted, primary, border, danger, and overlay. Components consume meanings rather than hard-coded colors. The selected mode is saved independently in AsyncStorage.

Date handling

Due dates use YYYY-MM-DD local date-only strings rather than UTC timestamps. A deadline represents a calendar day, not an instant, so this prevents timezone changes from shifting the displayed day. Utility functions validate real dates, format relative labels, and sort without mutating source arrays.

Component boundaries

Component	Reason it exists
TaskItem	One task card: completion, metadata, edit, delete.
TaskControls	Filter counts and current sort affordance.
SortSheet	Focused modal selection for sort order.
VoiceInputModal	Voice state presentation independent of recording logic.
FloatingActionButton	Primary voice entry point and listening feedback.
EmptyState	Purposeful no-task and no-result states.

8. Codebase map

Path	What to say in the interview
App.tsx	Composes providers, navigation theme, and status bar.
src/screens/TaskListScreen.tsx	Orchestrates derived list state and user commands.
src/screens/TaskFormScreen.tsx	Reusable controlled form for add and edit.
src/context/TaskContext.tsx	Application API plus persistence effects.
src/context/taskReducer.ts	Pure, deterministic state transitions.
src/hooks/useVoiceInput.ts	Recording/processing state machine.
src/services/taskStorage.ts	Runtime validation and AsyncStorage serialization.
src/services/voiceTaskApi.ts	Multipart client and response normalization.
src/utils/taskDates.ts	Date-only rules, labels, and stable sorting.
src/utils/normaliseVoiceTasks.ts	Trust boundary for AI-produced task data.
src/constants/theme.ts	Semantic design tokens for two themes.
server/src	Secure proxy and AI orchestration.
tests	Pure business-rule coverage.

How to trace a feature

EXAMPLE: COMPLETE A TASK TaskItem onToggle → TaskListScreen callback → TaskContext.toggleTask → TOGGLE_TASK reducer action → new tasks array → saveTasks effect → AsyncStorage.

EXAMPLE: EDIT A VOICE TASK TaskItem Edit → EditTask route with taskId → TaskFormScreen prefill → updateTask → UPDATE_TASK → source remains voice → persisted state.

9. Testing, quality, and build decisions

Automated checks

Command	What it protects
npm run typecheck	Mobile TypeScript correctness.
npm --prefix server run typecheck	Backend TypeScript correctness.
npm test	Reducer, validation, normalization, and date rules.
npm run lint	Code-quality rules with zero warnings allowed.
npm run format:check	Consistent formatting across the repository.
npx expo-doctor	Expo package and native-module compatibility.
npx expo export --platform ios	Production iOS bundling.

Important design choices and trade-offs

Decision	Why	Trade-off
Context + reducer	Explicit/testable and proportional	Would need evolution for a much larger app
One AsyncStorage array	Simple local-first persistence	No querying, sync, or conflict resolution
Backend proxy	Protects provider key	Voice needs network and server
Memory uploads	Simple for short interview recordings	Not suitable for large production media
Structured JSON schema	Predictable extraction contract	Provider/model must support schema output
Date-only strings	Correct calendar semantics	No time-of-day reminders
Transcript fallback	Never waste successful speech recognition	Fallback may be less granular

10. Likely interview questions and strong answers

Walk me through the architecture.

The app separates presentation, application state, device services, domain utilities, and a server-side AI boundary. Context exposes commands, the reducer owns deterministic transitions, AsyncStorage persists canonical tasks, and the backend protects credentials while orchestrating transcription and extraction.

Why use useReducer instead of multiple useState calls?

Task changes are related domain events. A reducer centralizes those transitions, prevents scattered mutation logic, and makes them easy to test without rendering UI.

Why not Redux?

The state surface is small and local to one provider. Redux would add dependencies and ceremony without a current scaling benefit. I would reconsider it if the app gained remote cache synchronization, many domains, or complex middleware.

How do you prevent data loss during startup?

Automatic saving is gated by both isHydrated and loadedSafely. A failed read must not cause the initial empty array to overwrite valid stored data.

Why validate data loaded from AsyncStorage?

Storage is an untrusted boundary: old versions, manual edits, or corruption can produce invalid JSON. The service parses safely and filters records against the current Task shape.

How do multiple voice tasks work?

Audio is transcribed first. A second model receives the transcript and returns strict structured JSON containing only explicit actions. The server trims and deduplicates; the client normalizes again before state insertion.

Why two AI calls?

Transcription answers what was said; extraction answers how many actions it contains. Separating concerns improves prompts, observability, provider flexibility, and fallback behavior.

What happens when extraction fails?

The transcript becomes a single task. This graceful fallback preserves useful user input while isolating failure of the optional structuring step.

Why is the API key on the server?

Anything in a React Native bundle can be inspected. A backend is the security boundary that keeps the key out of the application artifact.

How is editing implemented without duplicating UI?

AddTask and EditTask routes both render TaskFormScreen. The route determines initial values and submit behavior. UPDATE_TASK preserves immutable identity, source, completion, and creation timestamp.

Why store due dates as YYYY-MM-DD?

A due date is a local calendar day rather than a global instant. UTC timestamps can shift dates across timezones, while date-only values preserve the user's intent.

How do search and filters avoid corrupting state?

They are derived with useMemo from the canonical tasks array. sortTasks copies before sorting, so no derived operation mutates reducer state.

Interview questions (continued)

What makes the code modular?

UI components present state, hooks orchestrate device workflows, context exposes application commands, reducers transform state, services cross I/O boundaries, and utilities implement pure rules.

What would you improve for production?

Authentication, rate limiting, CORS restriction, streaming/object storage, telemetry, backend integration tests, end-to-end device tests, privacy/retention UX, and optional cloud synchronization.

What was the hardest problem?

The voice workflow combines permission handling, recorder lifecycle, file upload, two AI operations, untrusted outputs, fallbacks, and user-facing states. I reduced complexity by turning it into explicit states and narrow services.

How do you know it works?

The project passes strict app/server typechecks, 17 unit tests, zero-warning ESLint, Prettier, Expo Doctor, and an iOS production export; it was also tested on a physical iPhone with screenshots and a recording.

11. Interview demonstration script

Two-minute walkthrough

22. Open on the empty state. Say: “The app handles the no-data edge case and presents manual and voice entry points.”
23. Add a manual task with description and due date. Mention controlled inputs, validation, reducer dispatch, and AsyncStorage.
24. Complete the task and switch filters. Explain that visible results are derived rather than separately stored.
25. Edit the task. Point out that add/edit share TaskFormScreen.
26. Press the FAB and dictate two actions. Explain the recording state machine while processing.
27. Show the separate voice tasks and VOICE provenance label.
28. Switch theme and mention semantic tokens plus persisted preference.

Thirty-second architecture explanation

PRACTICE THIS The React Native app owns UI and local state. TaskContext exposes commands, a pure reducer performs transitions, and AsyncStorage persists the canonical task array. Voice recording is handled by a hook, uploaded through a typed client to Express, transcribed by Whisper, then converted into structured tasks by an LLM. The server protects credentials and falls back to the transcript if extraction fails.

If the demo network fails

Explain the expected error state rather than improvising: the mobile client maps network failures to a recovery message, existing tasks remain untouched, and manual task creation continues to work.

12. Final cheat sheet

Topic	Remember this sentence
State	Context exposes commands; reducer owns transitions.
Persistence	Load safely before allowing automatic saves.
Voice	Transcribe first, extract actions second, normalize at both boundaries.
Security	Secrets stay server-side; the mobile app gets only a backend URL.
Dates	Calendar days are stored as date-only values to avoid timezone drift.
Derived UI	Search, filter, counts, and sort are computed—not duplicated in state.
Editing	One TaskFormScreen serves AddTask and EditTask.
Quality	Strict types, tests, lint, format, Expo Doctor, and iOS export.
Trade-off	The design is intentionally proportional to a small local-first app.

Terms you should be able to define

- Controlled input: a form field whose displayed value comes from React state.
- Reducer: a pure function that returns the next state from current state and an action.
- Hydration: loading persisted state into the running application.
- Derived state: values computed from canonical state instead of separately stored.
- Trust boundary: a point where external data must be validated before use.
- Graceful degradation: retaining useful core behavior when an enhancement fails.
- Semantic theme token: a color named for its purpose, not its literal value.
- Structured output: model output constrained to a defined machine-readable schema.

FINAL ADVICE Be honest about trade-offs. A strong interview answer explains why a choice fits this app and when you would replace it—not that one tool is universally best.